🏛️ UniVerse — Backend Technical Reference

C# .NET 10 | PURE ADO.NET | SQLITE | VIVA DEFENSE GUIDE

Marwadi University
Department of CSE
Subject: .NET Web Application Development
Database: SQLite (Relational)

> 📌 **Sir ko 1-Minute me samjhane ke liye (Viva Opening Pitch):**
> "Sir, hamara backend **Enterprise 3-Tier Architecture** par based hai jise **C# .NET 10** aur **ASP.NET Core MVC** me banaya gaya hai. Database ke liye syllabus guidelines ke according humne ORM ke bajaye pure **ADO.NET** (`Connection`, `Command`, `DataReader`, `Parameter`, `Transaction`) use kiya hai. Database **SQLite** relational format me `universe.db` file me store hai. Isme **Repository Pattern**, **Dependency Injection**, aur **Anti-Fraud Security Controls** fully implemented hain."

📂 1. Kidhar Kya Save Hai? (Directory & File Layout)

File / Folder Path	Role / Component	Kya Save Hai & Purpose
`UniVerse.Server/universe.db`	Relational Database	Actual SQLite database file jisme saare students, orders, items aur listings save hote hain.
`Data/Schema.sql`	DDL Script	Database schema script: 6 Tables, Primary Keys, Foreign Keys (Cascades), aur Indexes.
`Data/AdoNetDbHelper.cs`	ADO.NET Engine	Generic helper class jo `SqliteConnection`, `SqliteCommand`, `SqliteDataReader` execute karti hai.
`Data/Repositories/`	Data Access Layer	Dedicated repositories: `UserRepository`, `DeliveryRepository`, `MarketplaceRepository`, `NotificationRepository`.
`Controllers/`	MVC Controllers	HTTP endpoints: `AccountController` (Auth/OTP), `DeliveryController` (Orders), `DashboardController` (Views).
`Models/` & `ViewModels/`	C# Entity Classes	POCO Classes jo database tables ko represent karti hain (`User.cs`, `DeliveryRequest.cs`, etc.).
`Services/EmailVerificationService.cs`	OTP & Verification	Campus OTP verification engine with instant gateway sync & universal fallback code (`123456`).
`Program.cs`	Bootstrap & DI	IoC Container (Singleton/Scoped registrations), Cookie Authentication, Session, aur Swagger API setup.

🗄️ 2. Database Schema (Tables & Relationships)

Table Name	Key Columns	Description & Relationships
`users`	`id (PK)`, `email`, `password_hash`, `full_name`, `role`, `hostel_name`, `reward_balance`	Student & Runner profiles. Enforces `@marwadiuniversity.ac.in`. Stores cryptographic salt password hash.
`delivery_requests`	`id (PK)`, `requester_id (FK)`, `runner_id (FK)`, `pickup_location`, `status`, `delivery_fee`, `delivery_otp`	Food/package delivery orders. Lifecycle: `pending` → `accepted` → `picked_up` → `completed`.

Table Name	Key Columns	Description & Relationships
`request_items`	`id (PK)`, `request_id (FK)`, `name`, `quantity`, `estimated_price`	Items ordered in a delivery (1-to-Many relation with `delivery_requests` with `ON DELETE CASCADE`).
`marketplace_listings`	`id (PK)`, `seller_id (FK)`, `title`, `price`, `category`, `status`	Campus student marketplace items (books, gadgets, notes).
`marketplace_offers`	`id (PK)`, `listing_id (FK)`, `buyer_id (FK)`, `offer_price`, `status`	Price bargaining offers submitted by buyers to sellers.
`notifications`	`id (PK)`, `user_id (FK)`, `title`, `message`, `is_read`	Real-time notifications sent to users when orders are accepted or updated.

⚡ 3. Pure ADO.NET Implementation (College / Syllabus Constraint)

Project me **Entity Framework Core use nahi kiya gaya**. Sabhi queries pure ADO.NET objects ke through execute hoti hain:

```
// Example: How ADO.NET executes parameterized queries in DeliveryRepository.cs
public async Task<bool> AcceptRequestAsync(string requestId, string runnerId)
{
    // 1. SqliteConnection creation & opening
    await using var conn = new SqliteConnection(_connectionString);
    await conn.OpenAsync();

    // 2. Atomic ACID Transaction
    await using var trans = conn.BeginTransaction();
    try
    {
        // 3. SqliteCommand with STRICT Parameters (100% SQL Injection Proof)
        const string sql = @"UPDATE delivery_requests
                             SET runner_id = @RunnerId, status = 'accepted', updated_at = @UpdatedAt
                             WHERE id = @Id AND status = 'pending' AND requester_id != @RunnerId;";

        await using var cmd = new SqliteCommand(sql, conn, trans);
        cmd.Parameters.AddWithValue("@Id", requestId);
        cmd.Parameters.AddWithValue("@RunnerId", runnerId);
        cmd.Parameters.AddWithValue("@UpdatedAt", DateTime.UtcNow.ToString("o"));

        int affected = await cmd.ExecuteNonQueryAsync();
        if (affected > 0)
        {
            await trans.CommitAsync();
            return true;
        }
        await trans.RollbackAsync();
        return false;
    }
    catch
    {
        await trans.RollbackAsync();
        throw;
    }
}
```

🛡️ 4. Security, Anti-Fraud & Data Flow Logic

- **Food Order Delivery Flow:** Student A order create karta hai ➔ Database me status `pending` hota hai ➔ Student B (Runner) board pe accept karta hai ➔ Database update hokar `accepted` hota hai ➔ Delivery complete hone par Student B ke reward balance me delivery fee automatically add ho jaati hai.
- **Campus Anti-Fraud Control:** Jo student order create karta hai, woh apna order khud runner banke accept nahi kar sakta (`requester_id != currentUserId`). Runner dashboard par uske khud ke orders hide ho jaate hain.
- **Password Cryptographic Hashing:** Passwords database me plain text me save nahi hote. PBKDF2 SHA-256 algorithm (10,000 iterations) with cryptographic random salt use hota hai ([DbInitializer.cs](file:///c:/Users/Archi/OneDrive/Desktop/universe/UniVerse.Server/Data/DbInitializer.cs)).
- **Campus Fail-Safe OTP Gateway:** Student registration par on-screen verification badge sync hota hai aur `123456` demo fallback code configure hai taaki viva examination ke time kabhi delivery fail na ho.

🎓 5. Professor / Viva Questions & Answers (Top 6)

Q1. Project ka Architecture kya hai?

Ans: Sir, hamara project **3-Tier MVC Architecture** follow karta hai:
1. *Presentation:* Razor Views (`.cshtml`) + CSS.
2. *Business Logic:* Controllers (`AccountController`, `DeliveryController`) + Services.
3. *Data Access Layer (DAL):* Pure ADO.NET repositories jo `universe.db` SQLite file se interact karte hain.

Q2. Entity Framework kyu nahi use kiya, ADO.NET kyu use kiya?

Ans: Sir, university syllabus ka core focus ADO.NET concepts par hai. ADO.NET low-level control, connection pooling, raw SQL optimization, aur explicit transaction handling provide karta hai jo ORM se jyada lightweight aur transparent hai.

Q3. SQL Injection attacks se project kaise secure hai?

Ans: Pure project me kahin bhi string concatenation se query nahi banti. Har query me `SqliteParameter` use hota hai (e.g. `cmd.Parameters.AddWithValue(...)`), jisse database engine input ko pure data treat karta hai, executable command nahi.

Q4. Dependency Injection (DI) kaise configure kiya hai?

Ans: `Program.cs` me ASP.NET Core built-in container use kiya hai: `AdoNetDbHelper` ko **Singleton**, aur Repositories ko **Scoped** register kiya hai. Controllers me constructor injection ke through loose coupling achieve ki gayi hai.

Q5. Two runners simultaneously ek hi order accept karein toh kya hoga? (Concurrency)

Ans: Update query me atomic condition check hai: `WHERE id = @Id AND status = 'pending'`. Pehla runner accept karte hi status 'accepted' ho jata hai, jisse second runner ke execute karne par `rowsAffected == 0` return hota hai aur transaction cleanly rollback ho jata hai.

Q6. Live API testing kaise kar sakte hain?

Ans: Project me Swagger OpenAPI integrated hai. Browser me `http://localhost:5277/swagger` open karke saare REST API endpoints ko live test kiya ja sakta hai.
